Università degli studi di Urbino “Carlo Bo”

Corso di Laurea L-31

(Informatica Applicata)

Progetto Architettura degli Elaboratori

Docente: Alessandro Bogliolo

Two Sum

Anno accademico 2024/2025

Studenti:

Andrea Pedini, N. 322918

Mattia Ruggieri, N. 322790

Two Sum

Specifica

Scopo del progetto

Lo scopo del progetto è quello di scrivere un segmento di codice Assembly che computi il seguente problema: dato in input un array di numeri interi e un intero “target”, se all’interno dell’array sono presenti due numeri la cui somma sia uguale al numero target, restituire come valore di ritorno la coppia delle posizioni dei due numeri.

Specifica algoritmica

```c
#include <stdio.h>

int main() {
    int size = 10;
    int target = 5;
    int result[2];
    int nums[size];
    int found = 0;

    for(int i = 0; i < size && !found; i++) {
        for(int j = i + 1; j < size && !found; j++) {
            if(nums[i] + nums[j] == target) {
                result[0] = i;
                result[1] = j;
                found = 1;
            }
        }
    }

    return 0;
}
```

File: TwoSum.c

Benchmark

Architettura di riferimento

L’architettura di riferimento per le simulazioni del progetto sono le seguenti:

- Code Address Bus: 10
- Data Address Bus: 10
- FP Addition Latency: 4
- Multiplier Latency: 7
- Division latency: 24
- Data forwarding: Attivo

Tipo di dati

I dati ai quali applicare il codice sono numeri interi con segno. Nella prima versione del codice non vi sono restrizioni sulla dimensione dell'array.

Caso Ottimo:

I due numeri la cui somma è uguale al valore cercato sono i primi due dell'array

Caso pessimo:

I due numeri la cui somma è ufguale al valore cercato sono gli ultimi due dell'array.

Prima implementazione in Assembly

```
01:     .data
02: nums:       .word   0,1,2,3,4,5,6,7
03: target:     .word   1
04: result1:    .word   -1
05: result2:    .word   -1
06: n:          .word   8
07:
08:     .text
09: start:
10:     LD      r1, n(r0)                   ; load number of elements (numsSize = n)
11:     LD      r2, target(r0)              ; load target value (value = target)
12:     DADDI   r10, r0, result1            ; load result1 (result1 = -1)
13:     DADDI   r11, r0, result2            ; load result2 (result2 = -1)
14:
15: startLoop1:
16:     DADDI   r3, r0, 0                   ; for(int i = 0...)
17:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
18:     DADDI   r4, r0, nums                ; pointer to first element in outer loop
19:
20: startLoop2:
21:     DADDI   r5, r3, 1                   ; for(int j = i + 1...)
22:     BEQ     r5, r1, end                 ; if(j == numsSize) jump to end
23:     DADDI   r6, r4, 8                   ; pointer to first element in inner loop
24:     LD      r7, 0(r4)                   ; load nums[i]
25:
26: loop2:
27:     LD      r8, 0(r6)                   ; load nums[j]
28:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
29:     BEQ     r9, r2, found               ; if(sum == target) -> found
30:     DADDI   r5, r5, 1                   ; else increment j
31:     BNE     r5, r1, loop2               ; if(j != n) -> inner loop
32:     DADDI   r6, r6, 8                   ; move to next element
33:
34: loop1:
35:     DADDI   r3, r3, 1                   ; else increment i
36:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
37:     DADDI   r4, r4, 8                   ; else move to next element
38:     J       startLoop2                  ; jump to startLoop2
39:
40: found:
41:     SD     r3, 0(r10)                   ; save i
42:     SD     r5, 0(r11)                   ; save j
43:
44: end:
45:     HALT                                ; end
```

File: TwoSum.s

Caso ottimo	Caso pessimo
Execution 26 Cycles 17 Instructions 1.529 Cycles Per Instruction (CPI) Stalls 4 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 1 Branch Taken Stall 0 Branch Misprediction Stalls	Execution 335 Cycles 206 Instructions 1.626 Cycles Per Instruction (CPI) Stalls 97 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 28 Branch Taken Stalls 0 Branch Misprediction Stalls

Versione 2

```
01:     .data
02: nums:       .word   0,1,2,3,4,5,6,7
03: target:     .word   1
04: result1:    .word   -1
05: result2:    .word   -1
06: n:          .word   8
07:
08:     .text
09: start:
10:     LD      r1, n(r0)                   ; load number of elements (numsSize = n)
11:     LD      r2, target(r0)              ; load target value (value = target)
12:     DADDI   r10, r0, result1            ; load result1 (result1 = -1)
13:     DADDI   r11, r0, result2            ; load result2 (result2 = -1)
14:
15: startLoop1:
16:     DADDI   r3, r0, 0                   ; for(int i = 0...)
17:     DADDI   r4, r0, nums                ; pointer to first element in outer loop
18:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1                   ; for(int j = i + 1...)
22:     DADDI   r6, r4, 8                   ; pointer to first element in inner loop
23:     BEQ     r5, r1, end                 ; if(j == numsSize) jump to end
24:     LD      r7, 0(r4)                   ; load nums[i]
25:
26: loop2:
27:     LD      r8, 0(r6)                   ; load nums[j]
28:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
29:     BEQ     r9, r2, found               ; if(sum == target) -> found
30:     DADDI   r5, r5, 1                   ; else increment j
31:     DADDI   r6, r6, 8                   ; move to next element
32:     BNE     r5, r1, loop2               ; if(j != n) -> inner loop
33:
34: loop1:
35:     DADDI   r3, r3, 1                   ; else increment i
36:     DADDI   r4, r4, 8                   ; else move to next element
37:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
38:     J       startLoop2                  ; jump to startLoop2
39:
40: found:
41:     SD     r3, 0(r10)                   ; save i
42:     SD     r5, 0(r11)                   ; save j
43:
44: end:
45:     HALT                                ; end
```

File: TwoSum1.s

Caso ottimo	Caso pessimo
Execution 24 Cycles 17 Instructions 1.412 Cycles Per Instruction (CPI) Stalls 2 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 1 Branch Taken Stall 0 Branch Misprediction Stalls	Execution 315 Cycles 227 Instructions 1.388 Cycles Per Instruction (CPI) Stalls 56 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 28 Branch Taken Stalls 0 Branch Misprediction Stalls

In questa versione è stato effettuato del lieve instruction reordering alle righe 17, 22, 31 e 36.

L'istruzione che sposta il puntatore al prossimo elemento dell'array a ogni ciclo è stata spostata prima del controllo se il contatore i o j abbia superato la dimensione massima dell'array.

In questo modo Guadagnamo un ciclo di clock che costituirebbe uno stallo di tipo RAW se effettuassimo immediatamente il controllo sul contatore.

Differenze

Versione 1	Versione 2
15: startLoop1: 16: DADDI r3, r0, 0 17: BEQ r3, r1, end 18: DADDI r4, r0, nums 20: startLoop2: 21: DADDI r5, r3, 1 22: BEQ r5, r1, end 23: DADDI r6, r4, 8 24: LD r7, 0(r4) 30: DADDI r5, r5, 1 32: BNE r5, r1, loop2 31: DADDI r6, r6, 8 34: loop1: 35: DADDI r3, r3, 1 36: BEQ r3, r1, end 37: DADDI r4, r4, 8 38: J startLoop2	15: startLoop1: 16: DADDI r3, r0, 0 17: DADDI r4, r0, nums 18: BEQ r3, r1, end 20: startLoop2: 21: DADDI r5, r3, 1 22: DADDI r6, r4, 8 23: BEQ r5, r1, end 24: LD r7, 0(r4) 30: DADDI r5, r5, 1 31: DADDI r6, r6, 8 32: BNE r5, r1, loop2 34: loop1: 35: DADDI r3, r3, 1 36: DADDI r4, r4, 8 37: BEQ r3, r1, end 38: J startLoop2

Versione 3

```
01:     .data
02: nums:       .word   0,1,2,3,4,5,6,7
03: target:     .word   13
04: result1:    .word   -1
05: result2:    .word   -1
06: n:          .word   8
07:
08:     .text
09: start:
10:     LD      r1, n(r0)                   ; load number of elements (numsSize = n)
11:     LD      r2, target(r0)              ; load target value (value = target)
12:     DADDI   r10, r0, result1            ; load result1 (result1 = -1)
13:     DADDI   r11, r0, result2            ; load result2 (result2 = -1)
14:
15: startLoop1:
16:     DADDI   r3, r0, 0                   ; for(int i = 0...)
17:     DADDI   r4, r0, nums                ; pointer to first element in outer loop
18:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1                   ; for(int j = i + 1...)
22:     LD      r7, 0(r4)                   ; load nums[i]
23:     BEQ     r5, r1, end                 ; if(j == numsSize) jump to end
24:
25: loop2:
26:     LD      r8, 8(r4)                   ; load nums[j]
27:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
28:     BEQ     r9, r2, found               ; if(sum == target) -> found
29:     DADDI   r5, r5, 1                   ; else increment j
30:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
31:
32:     LD      r8, 16(r4)                   ; load nums[j]
33:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
34:     BEQ     r9, r2, found               ; if(sum == target) -> found
35:     DADDI   r5, r5, 1                   ; else increment j
36:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
37:
38:     LD      r8, 24(r4)                   ; load nums[j]
39:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
40:     BEQ     r9, r2, found               ; if(sum == target) -> found
41:     DADDI   r5, r5, 1                   ; else increment j
42:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
43:
44:     LD      r8, 32(r4)                   ; load nums[j]
45:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
46:     BEQ     r9, r2, found               ; if(sum == target) -> found
47:     DADDI   r5, r5, 1                   ; else increment j
48:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
49:
50:     LD      r8, 40(r4)                   ; load nums[j]
51:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
52:     BEQ     r9, r2, found               ; if(sum == target) -> found
53:     DADDI   r5, r5, 1                   ; else increment j
54:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
55:
56:     LD      r8, 48(r4)                   ; load nums[j]
57:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
58:     BEQ     r9, r2, found               ; if(sum == target) -> found
59:     DADDI   r5, r5, 1                   ; else increment j
60:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
61:
62:     LD      r8, 56(r4)                   ; load nums[j]
63:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
64:     BEQ     r9, r2, found               ; if(sum == target) -> found
65:     DADDI   r5, r5, 1                   ; else increment j
66:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
67:
68: loop1:
69:     DADDI   r3, r3, 1                   ; else increment i
70:     DADDI   r4, r4, 8                   ; else move to next element
71:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
72:     J       startLoop2                  ; jump to startLoop2
73:
```

```
74: found:
75:     SD      r3, 0(r10)                  ; save i
76:     SD      r5, 0(r11)                  ; save j
77:
78: end:
79:     HALT                                ; end
80:
```

File: TwoSum2.s

Caso ottimo	Caso pessimo
Execution 23 Cycles 16 Instructions 1.438 Cycles Per Instruction (CPI) Stalls 2 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 1 Branch Taken Stall 0 Branch Misprediction Stalls	Execution 293 Cycles 193 Instructions 1.518 Cycles Per Instruction (CPI) Stalls 83 RAW Stalls 0 WAW Stalls 0 WAR Stalls 0 Structural Stalls 13 Branch Taken Stalls 0 Branch Misprediction Stalls

In questa versione siamo passati a effettuare il loop unrolling totale del ciclo innestato.

Con questa evoluzione la correttezza funzionale dell'algoritmo si riduce ad array la cui dimensione è pari a 8, in quanto con dimensioni maggiori non si effettuerebbero tutti i confronti necessari possibili tra le coppie di numeri.

Inoltre, dal momento in cui la dimensione della porzione di array da scorrere nel ciclo innestato si riduce di un'unità ad ogni ciclo esterno, nel codice del ciclo interno si controlla ad elemento processato se siamo giunti a fine array.

Effettuando questo loop unroll riduciamo considerevolmente il numero di stalli dovuti ai controlli sulle istruzioni di salto, tuttavia incrementiamo altrettanto notevolmente gli stalli RAW a causa del minor numero di istruzioni presenti nel corpo di ogni ciclo.

Differenze

Versione 2

```
15: startLoop1:
16:     DADDI   r3, r0, 0    ; i = 0
17:     DADDI   r4, r0, nums
18:     BEQ     r3, r1, end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1
22:     DADDI   r6, r4, 8
23:     BEQ     r5, r1, end
24:     LD      r7, 0(r4)
25:
26: loop2:
27:     LD      r8, 0(r6)
28:     DADD    r9, r7, r8
29:     BEQ     r9, r2, found
30:     DADDI   r5, r5, 1
31:     DADDI   r6, r6, 8
32:     BNE     r5, r1, loop2
33:
```

Versione 3

```
15: startLoop1:
16:     DADDI   r3, r0, 0
17:     DADDI   r4, r0, nums
18:     BEQ     r3, r1, end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1
22:     LD      r7, 0(r4)
23:     BEQ     r5, r1, end
24: - - - - - - - - - - - - - - - -
25:
26: loop2:
27:     LD      r8, 8(r4)
28:     DADD    r9, r7, r8
29:     BEQ     r9, r2, found
30:     DADDI   r5, r5, 1
31: - - - - - - - - - - - - - - - -
32:     BEQ     r5, r1, loop1
33:     . . .
... continues unrolled
```

Versione 4

```
01:     .data
02: nums:       .word   0,1,2,3,4,5,6,7
03: target:     .word   13
04: result1:    .word   -1
05: result2:    .word   -1
06: n:          .word   8
07:
08:     .text
09: start:
10:     LD      r1, n(r0)                   ; load number of elements (numsSize = n)
11:     LD      r2, target(r0)              ; load target value (value = target)
12:     DADDI   r10, r0, result1            ; load result1 (result1 = -1)
13:     DADDI   r11, r0, result2            ; load result2 (result2 = -1)
14:
15: startLoop1:
16:     DADDI   r3, r0, 0                   ; for(int i = 0...)
17:     DADDI   r4, r0, nums                ; pointer to first element in outer loop
18:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1                   ; for(int j = i + 1...)
22:     LD      r7, 0(r4)                   ; load nums[i]
23:     BEQ     r5, r1, end                 ; if(j == numsSize) jump to end
24:     DADDI   r6, r4, 8                   ; move to next element
25:
26: loop2:
27:     LD      r8, 0(r6)                   ; load nums[j]
28:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]       1
29:     BEQ     r9, r2, found               ; if(sum == target) -> found    2
30:     DADDI   r5, r5, 1                   ; else increment j
31:     DADDI   r6, r4, 8                   ; move to next element
32:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
33:
34:     LD      r8, 0(r6)                   ; load nums[j]
35:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
36:     BEQ     r9, r2, found               ; if(sum == target) -> found
37:     DADDI   r5, r5, 1                   ; else increment j
38:     DADDI   r6, r4, 8                   ; move to next element
39:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
40:
41:     LD      r8, 0(r6)                   ; load nums[j]
42:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
```

```
43:     BEQ     r9, r2, found               ; if(sum == target) -> found
44:     DADDI   r5, r5, 1                   ; else increment j
45:     DADDI   r6, r4, 8                   ; move to next element
46:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
47:
48:     LD      r8, 0(r6)                   ; load nums[j]
49:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
50:     BEQ     r9, r2, found               ; if(sum == target) -> found
51:     DADDI   r5, r5, 1                   ; else increment j
52:     DADDI   r6, r4, 8                   ; move to next element
53:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
54:
55:     LD      r8, 0(r6)                   ; load nums[j]
56:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
57:     BEQ     r9, r2, found               ; if(sum == target) -> found
58:     DADDI   r5, r5, 1                   ; else increment j
59:     DADDI   r6, r4, 8                   ; move to next element
60:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
61:
62:     LD      r8, 0(r6)                   ; load nums[j]
63:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
64:     BEQ     r9, r2, found               ; if(sum == target) -> found
65:     DADDI   r5, r5, 1                   ; else increment j
66:     DADDI   r6, r4, 8                   ; move to next element
67:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
68:
69:     LD      r8, 0(r6)                   ; load nums[j]
70:     DADD    r9, r7, r8                  ; sum = nums[i] + nums[j]
71:     BEQ     r9, r2, found               ; if(sum == target) -> found
72:     DADDI   r5, r5, 1                   ; else increment j
73:     DADDI   r6, r4, 8                   ; move to next element
74:     BEQ     r5, r1, loop1               ; if(j != n) -> inner loop
76: loop1:
77:     DADDI   r3, r3, 1                   ; else increment i
78:     DADDI   r4, r4, 8                   ; else move to next element
79:     BEQ     r3, r1, end                 ; if(i == numsSize) jump to end
80:     J       startLoop2                  ; jump to startLoop2
81:
82: found:
83:     SD      r3, 0(r10)                  ; save i
84:     SD      r5, 0(r11)                  ; save j
85:
86: end:
87:     HALT                                ; end
```

File: TwoSum3.s

Caso ottimo

```
Execution

24 Cycles
17 Instructions
1.412 Cycles Per Instruction (CPI)


Stalls

2 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
1 Branch Taken Stall
0 Branch Misprediction Stalls
```

Caso pessimo

```
Execution

300 Cycles
227 Instructions
1.322 Cycles Per Instruction (CPI)


Stalls

56 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
13 Branch Taken Stalls
0 Branch Misprediction Stalls
```

In questa versione abbiamo mantenuto il loop unrolling del ciclo innestato, tuttavia invece di caricare i dati dell'array scrivendo manualmente gli avanzamenti degli indirizzi di memoria, abbiamo reinserito l'uso di un puntatore che viene incrementato ad ogni ciclo.

In questo modo non vi sono perdite di performance, in quanto i cicli che impieghiamo nell'aggiornamento del puntatore, sarebbero stati stalli nella versione precedente. In questa versione infatti notiamo una notevole riduzione degli stalli RAW e del CPI medio.

Differenze

Versione 3

```
15: startLoop1:
16:     DADDI   r3, r0, 0
17:     DADDI   r4, r0, nums
18:     BEQ     r3, r1, end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1
22:     LD      r7, 0(r4)
23:     BEQ     r5, r1, end
24: - - - - - - - - - - - - - - -
25:
26: loop2:
27:     LD      r8, 8(r4)
28:     DADD    r9, r7, r8
29:     BEQ     r9, r2, found
30:     DADDI   r5, r5, 1
31: - - - - - - - - - - - - - - -
32:     BEQ     r5, r1, loop1
33:
```

Versione 4

```
15: startLoop1:
16:     DADDI   r3, r0, 0
17:     DADDI   r4, r0, nums
18:     BEQ     r3, r1, end
19:
20: startLoop2:
21:     DADDI   r5, r3, 1
22:     LD      r7, 0(r4)
23:     BEQ     r5, r1, end
24:     DADDI   r6, r4, 8
25:
26: loop2:
27:     LD      r8, 0(r6)
28:     DADD    r9, r7, r8
29:     BEQ     r9, r2, found
30:     DADDI   r5, r5, 1
31:     DADDI   r6, r4, 8
32:     BEQ     r5, r1, loop1
33:
```

Conclusioni

Versione 1

Caso ottimo

```
Execution
26 Cycles
17 Instructions
1.529 Cycles Per Instruction (CPI)

Stalls
4 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
1 Branch Taken Stall
0 Branch Misprediction Stalls
```

Caso pessimo

```
Execution
335 Cycles
206 Instructions
1.626 Cycles Per Instruction (CPI)

Stalls
97 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
28 Branch Taken Stalls
0 Branch Misprediction Stalls
```

Versione 4

Caso ottimo

```
Execution
24 Cycles
17 Instructions
1.412 Cycles Per Instruction (CPI)

Stalls
2 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
1 Branch Taken Stall
0 Branch Misprediction Stalls
```

Caso pessimo

```
Execution
300 Cycles
227 Instructions
1.322 Cycles Per Instruction (CPI)

Stalls
56 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
13 Branch Taken Stalls
0 Branch Misprediction Stalls
```